CS 0007: Introduction to Java Lecture 16

Nathan Ong
University of Pittsburgh
November 3, 2016

Main Portions of OOP

- Building a class
- Determining inter-class relationships
  - Subclasses
  - Superclasses
  - Ownership

Recap

A class needs these components:

- Class-level/Instance Variables
- Methods
- Constructor(s)

Car Example

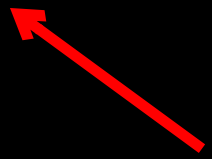
```
public class Car
{
     private static int nextIDNum = 0;
     private final int idNum;
     private Color color;
     private String licensePlate;

     public Car(Color color)
     {
     this.color = color;
     this.idNum = Car.nextIDNum;
     Car.nextIDNum++;
     …
     }//end constructor(Color)
     …
}//End class Car
```

Overloading Constructors

```
public class Car
{
 …
 public Car(Color color)
 {
 this.color = color;
 this.idNum = Car.nextIDNum;
 Car.nextIDNum++;
 …
 }//end constructor(Color)

 //default color is Color.RED
 public Car()
 {
 this(Color.RED);
 }//end constructor()
 …
}//End class Car
```



Referencing other constructors can be done with the keyword `this`

Functional Functions

- The most common functions are getters and setters
  - Getters retrieve the values of instance variables and return them
  - Setters alter the values of the instance variables
- Why use these functions rather than setting the variable to be public?
- Control!

Car Example

```
public class Car
{
     private static int nextIDNum = 0;
     private final int idNum;
     private Color color;
     private String licensePlate;

     public int getIDNum()
     {
     return this.idNum;
     }//end function()
     …
}//End class Car
```

Car Example

```
public class Car
{
     private static int nextIDNum = 0;
     private final int idNum;
     private Color color;
     private String licensePlate;

     public void setColor(Color color)
     {
     this.color = color;
     }//end function(Color)
     …
}//End class Car
```

Why Bother?

- Having these methods ensure you have full control over the instance variables
- Should you decide to change how the variable is accessed or altered, you only need to change the method.
- You only add them when you need them!

Inter-class Relationships

- Building a single class is useless. It needs to be used in context.
- There are two main relationships between classes
  - Sub/Super class
  - Ownership

Subclasses

- A subclass of a class is a more specific type of a class
- Also known as the child of a class
- For example, a tree would be a subclass of plant
- Subclasses are meant to expand upon the behavior of their more general parent class
- You can have several subclasses

Superclasses

- On the converse, a superclass is a more abstract and general version of a class
- Also known as the parent of a class
- Plant is the superclass of Tree
- You can only have one superclass

Declaring a Class to be a Subtype

- In order to declare a class's parent, use the keyword `extends`
- (Classes that do not have a parent class implicitly extend the Object class)

Inheritance

- A subclass inherits all non-private fields and methods
- Accessing these fields and methods requires the keyword `super`.
- (Note: you may also be able to access private fields with the parent class's getters and setters, which may be safer)

Restricting Access

- What if you want a field or method to be inherited, but still do not necessarily want it to be accessed by anyone?
- Use the keyword `protected`
- Remember, releasing your access to these fields and methods should only be done when necessary

Example

```
public class Vehicle
{
  private final int numWheels;
  …
  public boolean move(double dist)
  {
  return dist < this.numWheels*10.0;
  }//end function(double)
  …
}//End class Vehicle
```

Example A

```
public class Car extends Vehicle
{
  private int numPassengers;
  private double gasRemaining;
  …
  public boolean drive(double dist)
  {
  return (dist < this.numWheels*100.0) &&
      (this.gasRemaining > 0);
  }//end function(double)
  …
}//End class Plant
```

Example B

```
public class Car extends Vehicle
{
  private int numPassengers;
  private double gasRemaining;
  …
  public boolean drive(double dist)
  {
  return super.move(dist*10.0) &&
      (this.gasRemaining > 0);
  }//end function(double)
  …
}//End class Plant
```

Which Is Better?

- Example A totally ignores the move method (essentially its precursor), but is clearer
- Example B shows the movement lineage
- Either is okay, but both need comments!!!!

Regarding Constructors

- Constructors cannot be inherited, even though they are public
- However, you can, and should, use them from the subclass's constructor, to ensure complete initialization
- You can call the parent's constructor by using the keyword **`super`**

Calling Superclass Constructor

- Unlike calling an overloaded constructor of the same class, calling a superclass constructor does not need to be the only line of code
- However, it must still be the first line of code in the constructor

Example

```
public class Vehicle
{
  private final int numWheels;
  …
  public Vehicle(int numWheels)
  {
  this.numWheels = numWheels;
  …
  }//end function(double)
  …
}//End class Vehicle
```

Example

```
public class Car extends Vehicle
{
 private int numPassengers;
 private double gasRemaining;
 …
 public Car(int numPassengers, double gasRemaining)
 {
 super(4);
 this.numPassengers = numPassengers;
 this.gasRemaining = gasRemaining;
 }//end constructor(int,double)
 …
}//End class Plant
```

What If the Superclass's Method Definition Does Not Fit the Class's?

- We can override the definition by using the exact same function header
- The line directly above function header must have "`@Override`" only
- As an example, our Car class should probably use the original "move" method, rather than creating an entirely new "drive" method that does something similar

Reminder

```
public class Car extends Vehicle
{
  private int numPassengers;
  private double gasRemaining;
  …
  public boolean drive(double dist)
  {
  return super.move(dist*10.0) &&
      (this.gasRemaining > 0);
  }//end function(double)
  …
}//End class Plant
```

Override

```
public class Car extends Vehicle
{
  private int numPassengers;
  private double gasRemaining;
  …
  @Override
  public boolean move(double dist)
  {
  return super.move(dist*10.0) &&
      (this.gasRemaining > 0);
  }//end function(double)
  …
}//End class Plant
```

Overriding

- I can still reference the original method using the `super` keyword
- Unless your overridden method totally discards the functionality of the superclass's method, you should call the superclass's method at least once

Note

- You cannot override constructors

Ownership

- Ownership is simply one class using another as a property
- A Car may have an Engine, a Radio, and an Owner, depending on what you need
- A Car’s superclass and subclasses certainly are not Engine, Radio, and Owner

Using the Classes

```
public class Garage
{
 private Vehicle[] vehicles;
 …
 public Garage(int capacity)
 {
 this.vehicles = new Vehicle[capacity];
 for(int i = 0; i < vehicles.length; i++)
 {
 vehicles[i] = null;
 }
 …
 }//end constructor(int)
 …
}//End class Garage
```

Using the Classes

```
public class Garage
{
 …
 public boolean addVehicle(Vehicle vehicle)
 {
 //Note: there is a better way!
 for(int i = 0; i < vehicles.length; i++)
 {
 if(vehicles[i] == null)
 {
 vehicles[i] = vehicle;
 return true;
 }
 }
 return false;
 }//end method(Vehicle)
}//End class Garage
```

Using the Classes

```
public class JayLeno
{
 public static void main(String[] args)
 {
 Garage garage = new Garage(100);
 Car ferrari = new Car(5,100.0);
 garage.addVehicle(ferrari);
 ferrari.move();//??
 ferrari.drive();//??
 System.out.println("I have so many cars");
 }//end method main
}//End class JayLeno
```

QUESTIONS?